Bilgisayar Programlama

Karar ve Kontrol Yapıları

İçindekiler

1. Program Akış Kontrolü
   - Doğru/Yanlış (True/False) Kavramı
   - `if` ve `if-else` Karar Yapıları
   - İç İçe `if-else` Kullanımı
2. Gelişmiş Seçim Yapıları
   - `else-if` Merdiveni
   - Ternary (`? :`) Operatörü
   - `switch-case` ve Düşüş (Fall-through) Teorisi
3. Uygulamalar

BÖLÜM 1

Karar ve Kontrol Yapıları

1.1. Program Akış Kontrolüne Giriş

Yazılım geliştirme sürecinde, algoritmaların yürütülme akışı üç temel kontrol mekanizması ile yönetilir:

1. **Sıralı Akış (Sequential)**: Varsayılan yürütme modelidir. Komutlar, kaynak kodda yazıldıkları sıraya göre yukarıdan aşağıya doğru ardışık olarak işlenir.

2. **Seçimli Akış (Selection/Branching)**: Programın belirli bir mantıksal koşulun doğruluğuna bağlı olarak farklı kod bloklarına dallanmasını sağlar (`if` , `if-else` , `switch`).

3. **Tekrarlı Akış (Repetition/Looping)**: Belirli bir kod bloğunun, koşul sağlandığı sürece döngüsel olarak çalıştırılmasıdır (`for` , `while` , `do-while`).

Seçimli kontrol yapıları, algoritmaların dinamik kararlar alabilmesine olanak tanıyarak yazılımın farklı senaryolara uyum sağlamasını mümkün kılar.

1.2. C Dilinde Mantıksal (Boolean) Durumlar: İkilik Mantık

C dilinin erken standartlarında özel bir mantıksal (boolean) veri tipi bulunmamaktaydı (C99 standartları ile birlikte `<stdbool.h>` kütüphanesi ve `_Bool` tipi eklenmiştir). Ancak dilin mantıksal değerlendirme altyapısı şu evrensel tabana dayanır:

Mantıksal Değerlendirme Kuralı:

- **0 (Sıfır) Değeri**: Mantıksal olarak kesinlikle **Yanlış (False)** kabul edilir.
- **Sıfır Harici Her Türlü Değer**: (Pozitif veya negatif fark etmeksizin) mantıksal olarak **Doğru (True)** kabul edilir.

```c
int x = 5;
if (x) {  // x'in değeri 5'tir. Sıfırdan farklı olduğu için mantıksal olarak DOĞRU (true)
    printf("Bu komut satırı işlenecektir.\n");
}

int y = 0;
if (y) {  // y'nin değeri 0'dır. Mantıksal olarak YANLIŞ (false) kabul edildiğinden bu blo
    printf("Bu işlem atlanacaktır.\n");
}
```

Literatürde Geleneksel Tanımlama

Modern C standartları öncesinde veya geriye dönük uyumluluk gerektiren durumlarda, mantıksal değerler makro tanımlamaları (preprocessor directives) ile ifade edilirdi:

```
#define TRUE 1
#define FALSE 0
```

1.3. Tek Yönlü Karar Yapısı: `if` Deyimi (if Statement)

`if` deyimi, C dilindeki en ilkel ve temel seçimli kontrol yapısıdır. Belirtilen koşul ifadesini test eder; koşul sonucunun "doğru" (true) olması durumunda kendisine bağlı kod bloğunu (scope) çalıştırır. Sonuç "yanlış" (false) ise, mevcut blok tamamen atlanır.

Sözdizimsel Yapı (Syntax):

```
if (koşul_ifadesi) {
    // Koşul DOĞRU ise çalıştırılacak olan komut dizisi
} /* YANLIŞ ise blok atlanır ve program sonraki komuttan devam eder */
```

Tek Satırlık İfadelerde Kullanım: Eğer `if` deyimine yalnızca tek bir komut bağlıysa, blok sınırlarını belirten süslü parantezlerin `{ }` kullanımı isteğe bağlıdır (opsiyoneldir).

```
if (koşul_ifadesi)
    tek_bir_komut_calisir; // Süslü parantez olmadan kullanım
```

Akılda Tutulması Gereken Kapsam (Scope) Kuralı: Süslü parantez kullanılmadığında, yalnızca izleyen *ilk komut satırı* koşula bağlı kabul edilir. Koddaki girinti (indentation) görsel bir düzen sağlasa da derleyici açısından anlamsızdır.

```
if (x > 5)
    printf("Satır 1: x değişkeni 5'ten büyüktür.\n");  // `if` bloğuna aittir.
    printf("Satır 2: Bu satır her zaman çalışır!\n");  // Koşuldan KESİNLİKLE bağımsızdır.
```

1.4. Çift Yönlü Karar Yapısı: `if-else` Deyimi (if-else Statement)

Algoritmanın tek bir koşul üzerinden birbirini dışlayan iki farklı mantıksal yola ayrılması (bifurcation) gerektiğinde `if-else` yapısı kullanılır. Koşul doğru ise birinci blok (if bloğu), yanlış ise alternatif blok (else bloğu) çalıştırılır.

Sözdizimsel Yapı:

```
if (koşul_ifadesi) {
    // Koşul DOĞRU (true) ise yürütülecek mutlak blok
} else {
    // Koşul YANLIŞ (false) ise yürütülecek alternatif blok
}
```

Pratik Örnek: Akademik başarı durumunun değerlendirilmesi.

```
int basari_notu = 65;

if (basari_notu >= 60) {
    printf("Öğrenci dersten BAŞARILI olmuştur.\n");
} else {
    printf("Öğrenci dersten BAŞARISIZ olmuştur.\n");
}
```

1.5. İç İçe Geçmiş Karar Yapıları (Nested if-else)

Karmaşık karar ağaçlarını (decision trees) modellemek amacıyla, bağımsız bir `if` veya `else` gövdesinin içerisine alt seviye bir `if-else` hiyerarşisinin tanımlanmasıdır.

```
if (kosul_A) {
    if (kosul_B) {
        // Hem Koşul A, hem de Koşul B eşzamanlı sağlandığında çalışır.
    } else {
        // Koşul A sağlanmış, varlığında Koşul B sağlanmamışsa çalışır.
    }
} else {
    if (kosul_C) {
        // İlk başta Koşul A sağlanmamış, fakat Koşul C sağlanmışsa çalışır.
    }
}
```

Sözdizimi Hatası Riski: Sarkan else (Dangling else) Problemi C dilinin sözdizimi kuralları gereğince, blok parantezlerinin eksik olduğu durumlarda bir `else` ifadesi, daima kendinden önceki *en yakın boşlukta olan ve eşleşmemiş ilk* `if` deyimine bağlanır. Bu durum programda mantıksal yıkımlara yol açabilir.

```
// SORUNLU DURUM (Gerçek davranış, görsel girintiden farklıdır):
if (x > 0)
    if (y > 0)
        printf("Hem x hem de y pozitiftir.\n");
else  // DİKKAT: Görsel olarak ilk if'e aitmiş gibi dursa da, İKİNCİ if'e bağlanır!
    printf("x pozitif değildir.\n");
```

Mimari Çözüm (Blok Sınırlarının Belirlenmesi): Kodun stabilitesi için kapsam operatörlerinin (süslü parantez) kullanılması zorunlu bir mühendislik standardı olarak kabul edilmelidir.

```
if (x > 0) { // Üst seviye if bloğu izole edildi
    if (y > 0)
        printf("Hem x hem de y pozitiftir.\n");
} else {     // else, güvenli bir biçimde üst seviye if ile eşleşti
    printf("x pozitif değildir.\n");
}
```

1.6. Çoklu Karar Yapısı: `else-if` Merdiveni (else-if Ladder)

Birbirini karşılıklı olarak dışlayan (mutually exclusive) çok sayıda koşulun öncelik sırasına göre değerlendirildiği senaryolarda `else-if` zinciri tercih edilir. Yapı yukarıdan aşağıya işlenir ve koşulu ilk sağlayan bloğun icrasının ardından tüm zincir terk edilir.

Sözdizimsel Yapı:

```
if (kosul_1) {
    // Sadece ve ilk olarak Koşul 1 DOĞRU ise çalışır.
} else if (kosul_2) {
    // Koşul 1 YANLIŞ, ancak Koşul 2 DOĞRU ise çalışır.
} else if (kosul_3) {
    // Üstteki tüm koşullar YANLIŞ, Koşul 3 DOĞRU ise çalışır.
} else {
    // Hiyerarşideki HİÇBİR koşul geçerli değilse tetiklenen varsayılan blok (default).
}
```

Kademeli Değerlendirme Örneği: Harf Notu Algoritması

```c
int sinav_skoru = 85;

if (sinav_skoru >= 90) {
    printf("Harf Notu: AA (Mükemmel)\n");
} else if (sinav_skoru >= 80) {
    // Skor 90'dan küçük VE 80'den büyüktür.
    printf("Harf Notu: BA (Çok İyi)\n");
} else if (sinav_skoru >= 70) {
    printf("Harf Notu: BB (İyi)\n");
} else if (sinav_skoru >= 60) {
    printf("Harf Notu: CB (Orta)\n");
} else if (sinav_skoru >= 50) {
    printf("Harf Notu: CC (Yeterli)\n");
} else {
    printf("Harf Notu: FF (Başarısız)\n");
}
// 85 değeri, ilk olarak (>=80) koşuluyla eşleşir. "BA" çıktısı verilir ve merdiven sonlan
```

1.7. Koşul Yönlendirme Operatörü: Üçlü (Ternary) Operatör `? :`

Ternary operatörü, karar mekanizmalarını tek satırda çözümleyen ve C dilinde üç operand (işlenen) alan yegâne işleçtir. Genellikle, kontrol akışından ziyade ifadelere dayalı (expression-based) asgari atama işlemlerinde `if-else` yapısının zarif ve kompakt bir alternatifi olarak kullanılır.

Genel Formülasyon (Sözdizimi):

```
1  (mantiksal_kosul) ? dogruysa_alinacak_deger : yanlissa_alinacak_deger;
```

Endüstriyel Pratikte Uygulamaları:

```c
// Klasik Örnek: Mutlak maksimum değerin dinamik olarak seçimi
int max_skor = (oyuncu1_skor > oyuncu2_skor) ? oyuncu1_skor : oyuncu2_skor;

// Gelişmiş Örnek: Fonksiyon argümanı olarak doğrudan entegrasyon
printf("Akademik Durum: %s\n", (ortalama >= 2.0) ? "Mezun Olabilir" : "Mezun Olamaz");
```

Mühendislik Standartları Bağlamında İkaz: Bu operatör kod kalabalığını azaltsa da; iç içe geçmiş (nested ternary) formda kullanılması, kod onarım sürecini (maintenance) ve okunabilirliği büyük ölçüde tahrip eder. Bu gibi durumlarda geleneksel `if-else` yapılarına sadık kalmak projenin verimi adına esastır.

1.8. switch Deyimi (switch Statement)

Bir ifadenin değerine göre birden fazla seçenek arasından seçim yapmak için kullanılır. Genellikle bir değişkenin belirli sabit değerlere eşit olup olmadığını kontrol etmek için idealdir. `if-else` merdivenlerine kıyasla çok sayıda eşitlik kontrolünün yapıldığı durumlarda daha temiz bir görünüm sunar.

switch - Sözdizimi

Sözdizimi:

```
1   switch (ifade) {
2       case sabit1:
3           // ifade == sabit1 ise çalışacak kod
4           break;
5       case sabit2:
6           // ifade == sabit2 ise çalışacak kod
7           break;
8       case sabit3:
9       case sabit4:  // birden fazla case aynı kod için birleştirilebilir
10          // ifade == sabit3 veya sabit4 ise çalışacak kod
11          break;
12      default:
13          // hiçbir case eşleşmezse çalışacak kod
14          // opsiyoneldir
15  }
```

switch - Önemli Kurallar

`switch` yapısını kullanırken dikkat edilmesi gereken teknik kısıtlamalar vardır:

1. **İfade Tipi:** `ifade` tamsayı (integral) tipinde olmalıdır (`int` , `char` , `enum`).
   - × `float` , `double` veya `char *` (string) kullanılamaz!
2. **Sabit Değerler:** `case` yanındaki değerler **derleme zamanında (compile-time)** bilinen sabitler olmalıdır (literal veya `#define` sabiti). Değişken olamaz.
3. **Zorunlu Olmayan break:** Her `case` bloğunun sonuna `break;` konulması teknik olarak zorunlu değildir, ama akışı kesmek için *genellikle* gereksinimdir.
4. **Opsiyonel default:** `default` bloğu herhangi bir `case` eşleşmediğinde çalışır, zorunlu değildir ve genel pratik olarak en sona yazılır.

Fall-through (Düşüş) Mekanizması

`switch` yapısındaki en önemli özelliklerden biri **"fall-through"** denilen davranıştır. Eğer bir `case` bloğunun sonunda `break;` kullanılmazsa, program o bloktan çıkmaz ve eşleşme olmasa dahi bir sonraki `case` bloğundaki kodları çalıştırmaya devam eder.

```
1   int sayi = 2;
2   switch (sayi) {
3       case 1:
4           printf("Bir ");
5       case 2:
6           printf("İki ");   // sayı=2 olduğu için akış buradan başlar
7       case 3:
8           printf("Üç ");    // break olmadığı için bu da çalışır!
9   }
10  // Çıktı: "İki Üç "
```

Fall-through'un Bilinçli Kullanımı

Bazen fall-through bir hata değil, **bilinçli bir tasarım tercihi** olabilir. Birden fazla `case` için aynı kodun çalıştırılması istendiğinde bilerek `break` konulmaz.

```
1   char harf = 'E';
2   switch (harf) {
3       case 'A':
4       case 'E':
5       case 'I':
6       case 'O':
7       case 'U':
8           printf("Sesli harf\n"); // A, E, I, O, U için ortak kod
9           break;
10      default:
11          printf("Sessiz harf\n");
12  }
```

1.9. if-else vs switch Karşılaştırması

Karar yapısı seçerken aşağıdaki temel farklılıkları göz önünde bulundurmalısınız:

Özellik	if-else Merdiveni	switch Deyimi
Koşul Tipi	Herhangi bir ifade (Mantıksal, ilişkisel)	Sadece tamsayı tipi (int, char, enum)
İşlem Tipi	Karmaşık koşullar (`&&` , `||` , `<` , `>`) desteklenir	Sadece eşitlik kontrolü (`==`) yapılabilir
Çoklu Dallanma	`else if` blokları ile uzar	`case` etiketleri ile gruplanır
Performans	Baştan sona sırayla test edilir, dallanma arttıkça yavaşlayabilir	Derleyici tarafından genellikle "Jump table" ile optimize edilir, çok sayıda dalda daha hızlıdır
Okunabilirlik	Eşitlik kontrollerinin çok olduğu durumda karmaşıklaşır	Eşitliklerin çok olduğu durumlarda çok daha temiz ve okunabilirdir

1.10. Sıçrama (Jump) İfadelerine Genel Bakış

Standart **karar ve seçim** (decision/selection) yapıları haricinde (yani `if`, `else`, `switch`, `?:` dışında), programın akışını **doğrudan (şartsız olarak)** değiştiren komutlar da vardır.

Bu *sıçrama* komutları akış yönünü manipüle eder:

1. `break`: İçinde bulunduğu `switch` veya döngü (loop) bloğundan anında çıkılmasını sağlar. (Bunu `switch` konusunda gördük).
2. `continue`: Sadece döngülerde kullanılır. Döngünün o anki turunu atlayıp, bir sonraki iterasyona (koşul kontrolüne) geçmesini sağlar.
3. `return`: İçinde çalışılan fonksiyondan çıkmayı ve çağrıldığı yere dönmeyi sağlar. Genellikle `main` fonksiyonunun sonunda (`return 0;`) veya hata durumunda erken çıkış için (early-exit) kullanılır.
4. `goto`: Program akışını kod içindeki belirli bir "etikete" (label) koşulsuz atlatır.

goto Deyimi (Unconditional Jump)

`goto`, programın belli bir yerine konumlandırılmış bir etiket tanımlayarak akışı doğrudan oraya yönlendirir. Karmaşık ve **"spagetti koda"** (okunması ve takibi zor) yol açabildiği için **modern programlamada kullanımı pek tavsiye edilmez**. Fakat "iç içe derin birden fazla döngüden çıkmak" veya "hata anında hafıza temizliği" (cleanup) gibi bazı spesifik senaryolarda nadiren de olsa kullanılabilir.

Sözdizimi:

```
1  goto etiket_adi;
2
3  // ... atlanan, çalıştırılmayacak kodlar ...
4
5  etiket_adi:
6  // program buradan çalışmaya devam eder
```

goto - Örnek Kullanım

Hata yönetimi yaparken kodun en sonuna gitmek için tasarlanmış bir örnek senaryo:

```c
#include <stdio.h>

int main() {
    int bolen = 0;

    if (bolen == 0) {
        printf("HATA: Sifira bolme hatasi tespit edildi!\n");
        goto hata_durumu;  // Hata etiketine atla
    }
    // ... Bu kisim atlanir (calismaz) ...
    printf("Bu normal calisma kodudur (Islem basarili).\n");
    // ... Diger kompleks algoritmalar ...
    return 0; // Basarili cikis

hata_durumu:  // Etiket tanimi (sonunda iki nokta ust uste vardir)
    printf("Hata yonetimi ve temizlik islemleri tamamlandi.\n");
    return 1; // Basarisiz cikis
}
```

1.11. Bölüm Özeti: Karar Yapıları ve Örnekler

Karar yapılarının ne zaman ve nasıl kullanılacağına dair kısa bir özet:

Yapı	Kullanım Amacı	Kısa Örnek
`if`	Tek bir şarta bağlı işlem yapılacaksa.	`if (yas >= 18) printf("Reşit");`
`if-else`	Şart doğruysa bir işlem, yanlışsa başka bir işlem yapılacaksa.	`if (not > 50) printf("Geçti"); else printf("Kaldı");`
`else-if`	Birbirine bağlı birden fazla şart sırayla kontrol edilecekse.	`if (x>0) .. else if (x<0) .. else ..`
`switch-case`	Bir değişkenin (tamsayı/char) birçok sabit değere eşitliği aranıyorsa.	`switch (gun) { case 1: printf("Pzt"); break; }`
`? :` (Ternary)	Basit bir `if-else` tek satırda değişkene değer atamak için kullanılacaksa.	`max = (a > b) ? a : b;`

1.12. Kısa Devre (Short-Circuit) Değerlendirmesi

C dilinde mantıksal `&&` (VE) ve `||` (VEYA) operatörleri **kısa devre** (short-circuit) mantığıyla çalışır. Bu, soldan sağa doğru değerlendirme yaparken sonucun kesinleştiği anda **diğer şartların (sağ tarafın) kontrol edilmemesi** anlamına gelir.

Örnek - `&&` (VE) Operatörü:

```
1   if (x != 0 && (y / x) > 5) {
2       // ...
3   }
```

- **Etkisi:** Eğer `x == 0` ise ilk koşul yanlıştır (`false`). İlk koşul yanlış olduğu için `VE` işleminin doğru olma ihtimali kalmamıştır. C dili **ikinci koşulu hiç değerlendirmez**. Bu sayede **sıfıra bölme (divide by zero) hatası** çalışma anında (runtime) engellenmiş olur!

Örnek - `||` (VEYA) Operatörü:

```
1  if (kullanici_admin_mi || sifre_dogru_mu()) {
2      // ...
3  }
```

- Etkisi: İlk koşul `true` ise sonuç kesinlikle `true` olacaktır. Dolayısıyla ağır bir işlem olabilen `sifre_dogru_mu()` fonksiyonu **hiç çalıştırılmaz** (performans kazancı sağlar).

1.13. Sık Yapılan Hatalar: Sınır İhlali (Off-by-One)

Karar yapılarında ve döngülerde, sınır koşullarının (`<` mi yoksa `<=` mi?) yanlış değerlendirilmesi çok sık karşılaşılan bir mantık hatasıdır.

Hatalı Senaryo (Öğrenci Notu Belirleme):

```
1   // Geçme notu 50 olsun.
2   if (not > 50) {
3       printf("Geçtiniz");
4   } else {
5       printf("Kaldınız");
6   }
```

- Sınır Hatası (Edge Case): Öğrenci tam 50 aldığında ne olur? `50 > 50` yanlış olduğu için `else` bloğuna girer ve öğrenci haksız yere kalır!
- Doğrusu: `if (not >= 50)` olmalıdır.

En İyi Pratik (Best Practice): Karar mekanizmalarınızı yazarken, sınır ve uç değerleri (boundary conditions) mutlaka tek tek test edin.

1.14. En İyi Pratikler: Sarkan else (Dangling else) ve Parantezler

Önceki bölümlerde "else bağlanma sorunu" olarak gördüğümüz durum, derleyicinin `else` ifadesini her zaman kendinden önceki en yakın `if`'e bağlaması kuralından doğar.

Spagetti Kod (Yanıltıcı Girinti):

```
if (giris_basarili)
    if (admin_yetkisi_var)
        sistemi_baslat();
else
    hata_ver(); // GİRİŞ BAŞARISIZ olduğunda çalışması bekleniyordu!
```

Kodun girintilemesi (indentation) yanıltıcıdır. `else`, içteki `if`'e aittir.

Altın Kural (Golden Rule):

Apple'ın meşhur bir güvenlik zafiyetinin sebebi (goto fail bug) süslü parantez eksikliğiydi. **Etkilediği kod tek satır bile olsa her zaman süslü parantez `{ }` kullanın!**

```
if (giris_basarili) {
    if (admin_yetkisi_var) {
        sistemi_baslat();
    }
} else {
    hata_ver();
}
```

BÖLÜM 2: UYGULAMALAR

2.1. Uygulama 1: Temel if-else ve Mantıksal Operatörler

Amaç: Basit karar yapılarını ve mantıksal operatörleri pekiştirmek.

```c
#include <stdio.h>

int main() {
    int yas;
    char cinsiyet;
    int ehliyet_yas = 18;

    printf("=== EHLIYET BASVURU KONTROLU ===\n\n");

    // Kullanıcıdan bilgi al
    printf("Yasinizi giriniz: ");
    scanf("%d", &yas);

    printf("Cinsiyetinizi giriniz (E/K): ");
    scanf(" %c", &cinsiyet);  // Boşluk önemli!

    printf("\n--- SONUÇ ---\n");
```

Kod Devamı...

```c
    // Temel if
    if (yas >= ehliyet_yas) {
        printf("Yas kriteri uygun: %d yas (minimum: %d)\n", yas, ehliyet_yas);
    }

    // if-else
    if (yas >= ehliyet_yas) {
        printf("Ehliyet alabilirsiniz.\n");
    } else {
        printf("Ehliyet alamazsiniz. %d yil daha beklemeniz gerekiyor.\n",
               ehliyet_yas - yas);
    }

    // Mantıksal operatörlerle birleştirilmiş koşullar
    if (yas >= ehliyet_yas && (cinsiyet == 'E' || cinsiyet == 'K')) {
        printf("Tum kriterler uygun. Basvuru yapabilirsiniz.\n");
    } else if (yas < ehliyet_yas) {
        printf("Yas kriterini saglamiyorsunuz.\n");
```

Kod Devamı...

```c
    } else if (cinsiyet != 'E' && cinsiyet != 'K') {
        printf("Gecersiz cinsiyet girdiniz.\n");
    }

    // if içinde başka if - iç içe yapı
    if (yas >= ehliyet_yas) {
        if (cinsiyet == 'E') {
            printf("Erkek adaylar icin ek belge gerekmez.\n");
        } else if (cinsiyet == 'K') {
            printf("Kadin adaylar icin ek form doldurmaniz gerekiyor.\n");
        }
    }

    return 0;
}
```

2.2. Uygulama 2: else-if Merdiveni ve Sıcaklık Uyarı Sistemi

Amaç: Çoklu koşulların else-if ile nasıl yönetildiğini anlamak.

```c
#include <stdio.h>

int main() {
    float sicaklik;
    char birim;
    float donusturulmus;

    printf("=== ENDUSTRIYEL SICAKLIK UYARI SISTEMI ===\n\n");

    // Kullanıcıdan sıcaklık ve birim al
    printf("Sicaklik degerini giriniz: ");
    scanf("%f", &sicaklik);

    printf("Birimi giriniz (C/F/K): ");
    scanf(" %c", &birim);

    // Tüm sıcaklıkları Celsius'a çevir
    if (birim == 'C' || birim == 'c') {
```

Kod Devamı...

```c
        donusturulmus = sicaklik;
        printf("Girilen deger: %.2f °C\n", sicaklik);
    } else if (birim == 'F' || birim == 'f') {
        donusturulmus = (sicaklik - 32) * 5 / 9;
        printf("Girilen deger: %.2f °F = %.2f °C\n", sicaklik, donusturulmus);
    } else if (birim == 'K' || birim == 'k') {
        donusturulmus = sicaklik - 273.15;
        printf("Girilen deger: %.2f K = %.2f °C\n", sicaklik, donusturulmus);
    } else {
        printf("Gecersiz birim! C, F veya K giriniz.\n");
        return 1;  // Programı hata ile sonlandır
    }

    // Sıcaklık uyarı seviyesini belirle (else-if merdiveni)
    printf("\n--- UYARI SEVIYESI ---\n");

    if (donusturulmus < -50) {
        printf("KRITIK UYARI: Aşırı düşük sıcaklık! Sistem donma tehlikesi!\n");
```

Kod Devamı...

```c
        printf("Acil durum prosedürü başlatılıyor...\n");
    } else if (donusturulmus < -20) {
        printf("UYARI: Çok düşük sıcaklık! Isıtıcılar devreye alınmalı.\n");
    } else if (donusturulmus < 0) {
        printf("DIKKAT: Sıcaklık donma noktasının altında. Donma riski var.\n");
    } else if (donusturulmus < 20) {
        printf("NORMAL: Sıcaklık normal aralıkta.\n");
    } else if (donusturulmus < 40) {
        printf("DIKKAT: Sıcaklık yükseliyor. Soğutma sistemini kontrol edin.\n");
    } else if (donusturulmus < 60) {
        printf("UYARI: Yüksek sıcaklık! Soğutma kapasitesi zorlanıyor.\n");
    } else {
        printf("KRITIK UYARI: Aşırı yüksek sıcaklık! Acil kapatma gerekiyor!\n");
    }

    // Ek öneriler
    if (donusturulmus < 0 || donusturulmus > 40) {
        printf("\nONERI: Sistem güvenliği için operatör bilgilendirilmelidir.\n");
```

Kod Devamı...

```
    }

    return 0;
}
```

4.3. Uygulama 3: switch-case ile Menü Yönetimi

Amaç: switch-case yapısını ve fall-through mekanizmasını anlamak.

```c
#include <stdio.h>

int main() {
    int secim;
    float sayi1, sayi2, sonuc;

    printf("================================\n");
    printf("   GELISMIS HESAP MAKINESI     \n");
    printf("================================\n\n");

    printf("1. Toplama\n");
    printf("2. Cikarma\n");
    printf("3. Carpma\n");
    printf("4. Bolme\n");
    printf("5. Mod (tamsayi bolumunden kalan)\n");
    printf("6. Karekok (ilk sayi icin)\n");
    printf("7. Cikis\n\n");
```

Kod Devamı...

```c
    printf("Seciminiz (1-7): ");
    scanf("%d", &secim);

    // switch ile menü yönetimi
    switch (secim) {
        case 1:  // Toplama
            printf("Iki sayi giriniz: ");
            scanf("%f %f", &sayi1, &sayi2);
            sonuc = sayi1 + sayi2;
            printf("Sonuc: %.2f + %.2f = %.2f\n", sayi1, sayi2, sonuc);
            break;

        case 2:  // Cikarma
            printf("Iki sayi giriniz: ");
            scanf("%f %f", &sayi1, &sayi2);
            sonuc = sayi1 - sayi2;
            printf("Sonuc: %.2f - %.2f = %.2f\n", sayi1, sayi2, sonuc);
            break;
```

Kod Devamı...

```c

        case 3:  // Carpma
            printf("Iki sayi giriniz: ");
            scanf("%f %f", &sayi1, &sayi2);
            sonuc = sayi1 * sayi2;
            printf("Sonuc: %.2f * %.2f = %.2f\n", sayi1, sayi2, sonuc);
            break;

        case 4:  // Bolme
            printf("Iki sayi giriniz (bolunen bolen): ");
            scanf("%f %f", &sayi1, &sayi2);
            if (sayi2 != 0) {
                sonuc = sayi1 / sayi2;
                printf("Sonuc: %.2f / %.2f = %.2f\n", sayi1, sayi2, sonuc);
            } else {
                printf("HATA: Bolen sifir olamaz!\n");
            }
            break;
```

Kod Devamı...

```c

        case 5:  // Mod (tamsayılar için)
            printf("Iki tamsayi giriniz: ");
            scanf("%f %f", &sayi1, &sayi2);
            // Float'ları int'e çevir (basitlik için)
            if ((int)sayi2 != 0) {
                sonuc = (int)sayi1 % (int)sayi2;
                printf("Sonuc: %d %% %d = %d\n", (int)sayi1, (int)sayi2, (int)sonuc);
            } else {
                printf("HATA: Bolen sifir olamaz!\n");
            }
            break;

        case 6:  // Karekok (fall-through örneği)
            printf("Bir sayi giriniz: ");
            scanf("%f", &sayi1);
            if (sayi1 >= 0) {
                sonuc = sqrt(sayi1);  // <math.h> gerektirir
```

Kod Devamı...

```c
                printf("Karekok(%.2f) = %.2f\n", sayi1, sonuc);
            } else {
                printf("Negatif sayinin karekoku karmasiktir!\n");
            }
            break;
            // Burada break var, bu yüzden case 7'ye düşmez

        case 7:  // Çıkış
            printf("Program sonlandiriliyor...\n");
            break;

        default:  // Geçersiz seçim
            printf("Gecersiz secim! Lutfen 1-7 arasi bir deger giriniz.\n");
    }

    // Fall-through örneği - karakter bazlı menü
    printf("\n=== FALL-TROUGH ORNEGI ===\n");
    char opsiyon;
```

Kod Devamı...

```c
    printf("Bir harf giriniz (A, B, C, D, E): ");
    scanf(" %c", &opsiyon);

    switch (opsiyon) {
        case 'A':
        case 'B':
        case 'C':
            printf("A, B veya C sectiniz. Bunlar ana opsiyonlardir.\n");
            if (opsiyon == 'A') printf("A secenegi icin ekstra ayarlar yapiliyor...\n");
            break;  // Bu break, A,B,C'nin tamamı için

        case 'D':
            printf("D sectiniz.\n");
            // break yok - fall-through olacak!
        case 'E':
            printf("D veya E sectiniz. Bunlar yan opsiyonlardir.\n");
            break;
```

Kod Devamı...

```
        default:
            printf("Gecersiz harf!\n");
    }

    return 0;
}
```

4.4. Uygulama 4: Koşul Operatörü (Ternary) Kullanımı

Amaç: Ternary operatörün pratik kullanımını anlamak.

```c
#include <stdio.h>

int main() {
    int a, b;
    int max, min, mutlak;
    int sayi;

    printf("=== TERNARY OPERATOR ORNEKLERI ===\n\n");

    // Örnek 1: İki sayıdan büyük olanı bulma
    printf("Iki sayi giriniz: ");
    scanf("%d %d", &a, &b);

    max = (a > b) ? a : b;
    min = (a < b) ? a : b;

    printf("Sayilar: %d ve %d\n", a, b);
    printf("Buyuk olan: %d\n", max);
```

Kod Devamı...

```c
    printf("Kucuk olan: %d\n\n", min);

    // Örnek 2: Mutlak değer hesaplama
    printf("Bir sayi giriniz: ");
    scanf("%d", &sayi);

    mutlak = (sayi >= 0) ? sayi : -sayi;
    printf("|%d| = %d\n\n", sayi, mutlak);

    // Örnek 3: Çift/Tek kontrolü
    printf("%d sayisi %s\n", sayi, (sayi % 2 == 0) ? "cifttir" : "tektir");

    // Örnek 4: Sınır kontrolü
    int deger = 150;
    int sinirli = (deger > 100) ? 100 : (deger < 0) ? 0 : deger;
    printf("\nDeger: %d, Sinirli deger: %d\n", deger, sinirli);

    // Örnek 5: if-else vs ternary karşılaştırması
```

Kod Devamı...

```c
    int not = 75;
    char *harf_notu;

    // if-else ile
    if (not >= 90) harf_notu = "AA";
    else if (not >= 80) harf_notu = "BA";
    else if (not >= 70) harf_notu = "BB";
    else if (not >= 60) harf_notu = "CB";
    else if (not >= 50) harf_notu = "CC";
    else harf_notu = "FF";
    printf("if-else ile: %d -> %s\n", not, harf_notu);

    // ternary ile (karmaşık - önerilmez!)
    harf_notu = (not >= 90) ? "AA" :
                (not >= 80) ? "BA" :
                (not >= 70) ? "BB" :
                (not >= 60) ? "CB" :
                (not >= 50) ? "CC" : "FF";
```

Kod Devamı...

```
1       printf("ternary ile: %d -> %s\n", not, harf_notu);
2       printf("(Not: Karmasik ternary kullanimi okunabilirligi azaltir)\n");
3
4       return 0;
5   }
```

4.5. Uygulama 5: Bütünleşik Mühendislik Uygulaması - Motor Kontrol Sistemi

Amaç: Tüm karar yapılarını kullanarak gerçek bir mühendislik problemini çözmek.

```c
#include <stdio.h>

int main() {
    // Değişken tanımlamaları
    float sicaklik, akim, gerilim;
    float hiz, referans_hiz = 1500;  // rpm
    int acil_stop = 0;
    char calisma_modu;
    int hata_kodu = 0;

    printf("========================================\n");
    printf("   ENDUSTRIYEL MOTOR KONTROL SISTEMI   \n");
    printf("========================================\n\n");

    // Sensör verilerini oku (kullanıcıdan simüle)
    printf("Motor sicakligi (C): ");
    scanf("%f", &sicaklik);
```

Kod Devamı...

```c
    printf("Motor akimi (A): ");
    scanf("%f", &akim);

    printf("Motor gerilimi (V): ");
    scanf("%f", &gerilim);

    printf("Motor hizi (rpm): ");
    scanf("%f", &hiz);

    printf("Calisma modu (N:Normal, E:Eko, Y:Yuksek performans): ");
    scanf(" %c", &calisma_modu);

    printf("\n--- SISTEM ANALIZI ---\n");

    // 1. Acil durum kontrolleri (öncelikli)
    if (sicaklik > 85.0) {
        printf("KRITIK: Motor asiri sicak! (%.1f C)\n", sicaklik);
        acil_stop = 1;
```

Kod Devamı...

```c
        hata_kodu = 1;
    } else if (akim > 15.0) {
        printf("KRITIK: Asiri akim! (%.1f A)\n", akim);
        acil_stop = 1;
        hata_kodu = 2;
    } else if (gerilim < 180.0 || gerilim > 260.0) {
        printf("KRITIK: Gerilim aralik disi! (%.1f V)\n", gerilim);
        acil_stop = 1;
        hata_kodu = 3;
    }

    // Acil stop kontrolü
    if (acil_stop) {
        printf("\n!!! ACIL DURUM: Motor durduruluyor! Hata Kodu: %d !!!\n", hata_kodu);
        printf("Guvenli moda geciliyor...\n");
        return 1;  // Programı sonlandır
    }
```

Kod Devamı...

```c
    // 2. Uyarı seviyeleri (else-if merdiveni)
    printf("\n--- UYARI SEVIYELERI ---\n");

    if (sicaklik > 75.0) {
        printf("UYARI: Sicaklik yuksek (%.1f C). Sogutma fanini calistirin.\n", sicaklik);
    } else if (sicaklik > 60.0) {
        printf("DIKKAT: Sicaklik normalin ustunde (%.1f C).\n", sicaklik);
    } else if (sicaklik < 10.0) {
        printf("DIKKAT: Sicaklik dusuk (%.1f C). Isitma gerekebilir.\n", sicaklik);
    } else {
        printf("Sicaklik normal aralikta (%.1f C).\n", sicaklik);
    }

    // 3. Çalışma moduna göre hız kontrolü (switch ile)
    printf("\n--- CALISMA MODU ANALIZI ---\n");
    printf("Secilen mod: ");

    switch (calisma_modu) {
```

Kod Devamı...

```c
        case 'N':
        case 'n':
            printf("Normal Mod\n");
            if (hiz < referans_hiz * 0.9) {
                printf("Hiz dusuk (%.0f rpm). Voltaj arttiriliyor...\n", hiz);
            } else if (hiz > referans_hiz * 1.1) {
                printf("Hiz yuksek (%.0f rpm). Voltaj dusuruluyor...\n", hiz);
            } else {
                printf("Hiz normal (%.0f rpm).\n", hiz);
            }
            break;

        case 'E':
        case 'e':
            printf("Eko Mod (Enerji tasarrufu)\n");
            if (hiz > referans_hiz * 0.8) {
                printf("Eko mod icin hiz cok yuksek. Hiz dusuruluyor...\n");
            } else {
```

Kod Devamı...

```c
                printf("Hiz eko mod icin uygun.\n");
            }
            break;

        case 'Y':
        case 'y':
            printf("Yuksek Performans Modu\n");
            if (hiz < referans_hiz * 1.2) {
                printf("Hiz artiriliyor (talep: %.0f rpm, mevcut: %.0f rpm)\n",
                       referans_hiz * 1.2, hiz);
            } else {
                printf("Performans modunda calisiliyor.\n");
            }
            break;

        default:
            printf("TANIMSIZ MOD! Varsayilan normal mod kullaniliyor.\n");
    }
```

Kod Devamı...

```c

    // 4. Karmaşık karar yapısı - iç içe if'ler
    printf("\n--- VERIMLILIK ANALIZI ---\n");

    float verim = (akim > 0) ? (hiz * gerilim) / (akim * gerilim) * 100 : 0;

    if (verim > 85) {
        printf("Verim: %.1f%% - COK IYI\n", verim);

        if (calisma_modu == 'E' || calisma_modu == 'e') {
            printf("Eko modda yuksek verim! Tebrikler.\n");
        }
    } else if (verim > 70) {
        printf("Verim: %.1f%% - NORMAL\n", verim);

        // İç içe if-else
        if (sicaklik > 70) {
            printf("Verim dusukluğu sicakliktan kaynaklaniyor olabilir.\n");
```

Kod Devamı...

```c
        } else if (akim > 12) {
            printf("Verim dusukluğu yuksek akimdan kaynaklaniyor.\n");
        } else {
            printf("Verim normal sinirlarda.\n");
        }
    } else {
        printf("Verim: %.1f%% - DUSUK. Bakim gerekli!\n", verim);

        // Birden fazla koşulun mantıksal operatörlerle kontrolü
        if (sicaklik > 70 && akim > 12) {
            printf("MULTIPLE SORUN: Asiri sicaklik ve yuksek akim!\n");
        } else if (sicaklik > 70 || akim > 12) {
            printf("Tekil sorun tespit edildi.\n");
        }
    }

    // 5. Ternary operatör ile basit kararlar
    printf("\n--- ONERILER ---\n");
```

Kod Devamı...

```c

    char *bakim_durumu = (hiz < referans_hiz * 0.5 || sicaklik > 75) ?
                         "Acil bakim gerekli" :
                         (hiz < referans_hiz * 0.8) ? "Rutin bakim zamani" :
                         "Bakim gereksiz";
    printf("Bakim durumu: %s\n", bakim_durumu);

    float onerilen_akim = (calisma_modu == 'Y' || calisma_modu == 'y') ? 12.0 :
                          (calisma_modu == 'E' || calisma_modu == 'e') ? 8.0 : 10.0;
    printf("Onerilen maksimum akim: %.1f A\n", onerilen_akim);

    if (akim > onerilen_akim) {
        printf("UYARI: Akim onerilen degerin ustunde!\n");
    }

    printf("\n--- SISTEM DURUMU: NORMAL ---\n");

    return 0;
```

Kod Devamı...

```
}
```

4.6. Ödev ve Proje Bazlı Görevler (Pekiştirme İçin)

1. **Elektrik Faturası Hesaplayıcı**: Kullanıcıdan tüketilen kWh miktarını alın. Aşağıdaki tarifeye göre fatura tutarını hesaplayan bir program yazın:
   - 0-100 kWh: 1.5 TL/kWh
   - 101-200 kWh: 2.3 TL/kWh (ilk 100 kWh için 1.5, kalan için 2.3)
   - 201-300 kWh: 3.2 TL/kWh
   - 300+ kWh: 4.5 TL/kWh
   - Ayrıca, toplam tüketim 500 kWh'i geçerse ek olarak %20 vergi ekleyin

2. **Su Seviye Kontrol Sistemi**: Bir su deposunun seviyesini (0-100% arası) okuyan bir program yazın.
   - Seviye < %20: "ACİL: Su seviyesi çok düşük, pompa çalıştırılıyor"
   - Seviye %20-40: "UYARI: Su seviyesi düşük, pompa çalışıyor"
   - Seviye %40-70: "NORMAL: Su seviyesi normal"
   - Seviye %70-90: "UYARI: Su seviyesi yüksek, pompa durduruluyor"
   - Seviye > %90: "ACİL: Taşma riski! Vana açılıyor"
   - Ayrıca, seviye saatlik olarak %5'ten fazla değiştiyse ek uyarı verin

3. **Basit Bir ATM Uygulaması:**
   - Kullanıcıdan bir kart numarası (4 haneli) ve PIN (4 haneli) alın
   - Kart numarası 1234 ve PIN 5678 ise giriş başarılı olsun
   - Başarılı girişten sonra: Bakiye sorgulama, para çekme, para yatırma seçenekleri sunun (switch ile)
   - Para çekme işleminde bakiyeyi kontrol edin, yetersiz bakiye için uyarı verin
   - Her işlemden sonra güncel bakiyeyi gösterin
4. **Hata Ayıklama Egzersizi:** Aşağıdaki hatalı kod parçasını inceleyin ve tüm hataları bulup düzeltin:

```c
#include <stdio.h>

int main() {
    int x = 5, y = 10;

    if x > y
        printf("x y'den buyuk");
    else if (x = y)
        printf("x y'ye esit");
    else
        printf("x y'den kucuk");

    // switch hatası
    float sayi = 5.5;
    switch (sayi) {
        case 5.5:
            printf("Sayi 5.5");
            break;
```

Kod Devamı...

```
    }

    // else bağlanma hatası
    if (x > 0)
        if (y > 0)
            printf("Pozitif");
    else
        printf("Negatif");

    return 0;
}
```